

Engenharia de Requisitos para ML com GR4ML

Caderno do Grupo — JurisTriage PT

Disciplina: Engenharia de Software para IA e Frameworks Profundos

Projeto: JurisTriage PT

Grupo: Alessandro, Daniela, Gleicy, Gustavo, Helton, Luciana, Pedro e Sandro

Natureza: Prova de Conceito académica, não destinada a substituir profissionais do Direito nem a produzir aconselhamento jurídico.

*Nota de rigor: o repositório ainda não contém resultados finais de treino nem uma meta organizacional validada com utilizadores externos. Por isso, valores de Macro-F1, latência e impacto no trabalho são apresentados como **metas a validar**, nunca como resultados alcançados.*

1. Contexto do projeto — Fase 0

O JurisTriage PT é uma prova de conceito que pretende classificar automaticamente o **sentido normativo de acórdãos portugueses** em cinco categorias (**MANTIDA**, **REVOGADA**, **ANULADA**, **NAO_CONHECIDA** e **OUTRA**). O principal stakeholder operacional proposto é o **analista jurídico/investigador de jurisprudência**, que atualmente precisa de ler e categorizar manualmente grandes volumes de decisões para produzir análises agregadas. A dor central é o custo e a inconsistência dessa triagem manual. O sistema deve apoiar — e não substituir — a revisão humana, usando apenas os **descritores** e o **sumário**, com prevenção explícita de fuga de informação, proteção de dados e avaliação rigorosa em classes desequilibradas.

Âncora do projeto: ajudar analistas jurídicos e investigadores a organizar grandes coleções de jurisprudência por resultado processual, de forma reprodutível, auditável e suficientemente rápida para análise exploratória, sem usar campos que revelem diretamente a resposta.

2. Business View — Fase 1

2.1 Atores de negócio

Ator	Papel na organização/projeto
Analista jurídico / investigador de jurisprudência	Consulta e organiza decisões judiciais para análise temática, estatística ou académica. É o utilizador principal da classificação e dos resumos agregados.
Responsável técnico / Scrum Master	Garante requisitos, arquitetura, rastreabilidade, qualidade, integração e uso correto dos artefactos de ML.
Cientista/engenheiro de dados da equipa	Prepara o corpus, treina, avalia e monitoriza a qualidade técnica da solução.
Professor/banca académica	Patrocinador e avaliador da PoC; verifica se o projeto demonstra engenharia de software aplicada a IA, e não apenas uma métrica isolada.

2.2 Goals, indicadores e expectativas de qualidade

Ator	Goal	Indicador	Expectativas de qualidade
Analista jurídico / investigador	Reduzir o esforço de triagem inicial de acórdãos, recebendo uma categoria provável e a distribuição das cinco classes.	Tempo médio de triagem por acórdão e percentagem de documentos corretamente encaminhados para a categoria/revisão. Valores atuais e meta: a medir num estudo com utilizadores.	A classificação deve ser apresentada como apoio; os erros devem ser visíveis; a saída não pode parecer aconselhamento jurídico; deve ser possível rever casos ambíguos.
Responsável técnico	Garantir um pipeline íntegro, reproduzível e sem fuga de informação.	Percentagem de execuções com manifesto completo; testes aprovados; zero campos proibidos em X.	Artefactos de uma execução não podem ser misturados; a inferência deve usar a mesma composição do treino; as falhas pontuais não devem parar o lote.
Cientista/engenheiro de dados	Produzir um modelo que aprenda algo útil para todas as classes, apesar do desequilíbrio.	Macro-F1 do modelo, Macro-F1 da baseline de classe maioritária, perda por configuração e métricas por execução.	O modelo deve superar a baseline; o fit do TF-IDF deve ocorrer só no treino; resultados devem ser reproduzíveis e auditáveis.
Professor/banca	Avaliar uma PoC coerente de engenharia de software para IA.	Cobertura dos requisitos, modularidade, testes, NumPy, PyTorch, documentação e demonstração ponta a ponta.	O foco não é obter o modelo jurídico mais sofisticado; é demonstrar contratos, qualidade, ética, rastreabilidade e integração.

2.3 Decision Goals

- O **analista jurídico precisa decidir** que categoria usar como ponto de partida para organizar cada acórdão — **MANTIDA**, **REVOGADA**, **ANULADA**, **NAO_CONHECIDA**, **OUTRA** ou revisão manual — sempre que processa um novo documento ou lote.

- **O responsável técnico precisa decidir** se uma execução treinada pode ser aprovada para demonstração/inferência, depois de verificar integridade dos artefactos, fuga de informação, testes e comparação com a baseline.
- **O cientista de dados precisa decidir** qual configuração do classificador MLP deve ser conservada, após comparar configurações candidatas pelas métricas e restrições do projeto.

2.4 Question Goals

ID	Question Goal	Tipo	Tempo	Frequência
QG-01	Qual das cinco categorias melhor representa o sentido normativo deste novo acórdão, usando apenas descritores e sumário?	O quê	Presente/desconhecido	Por documento ou por lote
QG-02	Qual é a distribuição de probabilidade do modelo pelas cinco categorias e quais termos TF-IDF tiveram maior peso na entrada?	O quê/porquê aproximado	Presente	Por inferência
QG-03	O modelo supera a estratégia de prever sempre a classe maioritária em Macro-F1?	Comparação	Passado, sobre o teste	Por execução de treino
QG-04	A execução respeita as regras de fuga de informação, reprodutibilidade e alinhamento de artefactos?	Verificação	Presente	Por execução/entrega
QG-05	Qual configuração candidata oferece o melhor compromisso entre Macro-F1, simplicidade, reprodutibilidade e custo?	Qual	Futuro/seleção	Por experiência de treino

2.5 Diagrama da Business View

```

[Analista jurídico / investigador]
  |
  v
[GOAL: reduzir esforço e inconsistência da triagem inicial]
  |   Indicadores: tempo de triagem + encaminhamento correto
  |   Qualidades: apoio humano, transparência, privacidade
  |
  v
[DECISION GOAL: escolher categoria inicial ou revisão manual]
  |
  +--> [QG-01: qual das cinco classes?]
  +--> [QG-02: distribuição e termos relevantes?]

[Responsável técnico / Cientista de dados]
  |
  v
[GOAL: aprovar uma execução útil, reproduzível e sem fuga]
  |
  +--> [QG-03: supera baseline em Macro-F1?]
  +--> [QG-04: artefactos e regras estão íntegros?]
  +--> [QG-05: qual configuração selecionar?]

```

3. Analytics Design View — Fase 2

3.1 Tipo de análise

Question Goal	Tipo de análise	Justificação
QG-01	Preditiva	A classe verdadeira é desconhecida no momento da inferência e precisa de ser estimada a partir do texto permitido.
QG-02	Descritiva	Resume a saída do classificador e destaca os termos com maior peso TF-IDF; não constitui explicação jurídica causal.
QG-03	Descritiva/avaliativa	Compara o desempenho observado do modelo com uma baseline simples.
QG-04	Descritiva/diagnóstica	Verifica regras, artefactos e evidências de uma execução.
QG-05	Prescritiva	Recomenda qual configuração conservar com base em métricas e restrições.

3.2 Tarefas de ML e algoritmos candidatos

Meta analítica	Tarefa de ML	Algoritmos candidatos
Classificar o sentido normativo do acórdão	Classificação multiclasse supervisionada em cinco classes	1. Regra de palavras-chave/normalização como referência operacional; 2. baseline de classe maioritária; 3. Regressão Logística sobre TF-IDF (candidato conceptual, não implementado por restrição curricular); 4. MLP PyTorch sobre TF-IDF NumPy.
Descrever a incerteza da previsão	Distribuição multiclasse por softmax, sem afirmar calibração	Softmax dos logits do MLP; top termos pelo peso TF-IDF da entrada.
Selecionar configuração	Comparação experimental	MLP A com camada oculta e hiperparâmetros base; MLP B com hiperparâmetros/complexidade alternativos; seleção final por Macro-F1 de validação/teste e não apenas perda de treino.

Algoritmo escolhido

O algoritmo principal escolhido é um **Multilayer Perceptron (MLP) em PyTorch alimentado por TF-IDF implementado em NumPy**. Esta escolha decorre do escopo académico obrigatório: uso real de NumPy na engenharia de características, uso de PyTorch no treino, comparação de pelo menos duas configurações e execução offline. A baseline de classe maioritária permanece obrigatória para provar que a rede aprendeu algo útil.

3.3 Softgoals formalizados

ID	Softgoal	Origem
SG-01	Rigor em classes desequilibradas	A classe MANTIDA representa aproximadamente 52% das decisões não vazias; exatidão isolada seria enganadora.
SG-02	Ausência de fuga de informação	Regra do projeto: decisao_bruta , texto dispositivo, ECLI, URL, tribunal e texto_integral não podem entrar em X.
SG-03	Reprodutibilidade e rastreabilidade	Requisito técnico: semente 42, manifesto, configuração, pesos, vocabulário e métricas alinhados por execução.
SG-04	Privacidade e uso ético	O corpus pode conter dados pessoais; dados reais e artefactos identificáveis não devem sair do ambiente autorizado.
SG-05	Transparência e supervisão humana	A previsão não é decisão jurídica; a explicação é aproximada; casos ambíguos devem poder seguir para revisão.
SG-06	Robustez e portabilidade	Um documento problemático não deve interromper o lote; a PoC deve executar numa amostra pequena e num computador modesto.

3.4 Softgoals, métricas e metas

Softgoal	Métrica técnica	Meta/limite
SG-01 — Rigor em desequilíbrio	Macro-F1 do modelo e da baseline	Aceitação mínima: Macro-F1 do MLP > Macro-F1 da baseline no conjunto de teste. Meta numérica absoluta: a validar após experiência real.
SG-02 — Sem fuga	Auditoria dos campos usados em <code>Acordao.texto_caracteristicas()</code> e teste do vocabulário	100% das características derivadas apenas de descritores + sumario ; zero campos de CAMPOS_FUGA_INFORMACAO usados diretamente.
SG-03 — Reprodutibilidade	Semente registada; manifesto e ficheiros por id_execucao ; repetição controlada	Semente 42 e todos os caminhos essenciais presentes no manifesto; nenhuma mistura entre execuções.
SG-04 — Privacidade	Auditoria de Git/artefactos/serviços externos	Zero PDFs/JSONs reais, dados pessoais, segredos ou vocabulários identificáveis enviados para APIs externas ou versionados.
SG-05 — Transparência	Presença de distribuição, aviso e possibilidade de revisão	Toda a saída explicativa deve dizer que é automática e não constitui aconselhamento jurídico; probabilidades não devem ser apresentadas como confiança calibrada.
SG-06 — Robustez/portabilidade	Taxa de continuação após ficheiro inválido; execução em amostra	Falha isolada não interrompe o lote; fluxo demonstrável numa amostra de 10 documentos num computador modesto.

3.5 Matriz Algoritmo × Softgoals

Algoritmo	SG-01: Macro-F1	SG-02: fuga	SG-03: reprodutibilidade	SG-05: transparência	Conclusão
Classe maioritária	Macro-F1 baixo esperado; serve de baseline	Compatível, porque não usa texto	Muito simples e determinística	Muito transparente, mas não útil para classes minoritárias	Manter apenas como referência.
Regras por palavras-chave	Avaliação necessária; frágeis a variações linguísticas	Risco controlável se aplicadas só ao texto permitido	Reproduzíveis	Elevada legibilidade, mas manutenção difícil e propensa a exceções	Úteis para normalizar Y e como comparação, não como modelo final de X.
Regressão Logística + TF-IDF	Candidato potencialmente forte e interpretável	Compatível com X permitido	Reproduzível	Pesos por classe são inspecionáveis	Não escolhida no MVP por restrição/ênfase curricular em PyTorch.
MLP PyTorch + TF-IDF NumPy	Métrica principal a medir; deve superar baseline	Compatível se usar a composição canónica	Suporta semente, <code>state_dict</code> e manifesto	Menos interpretável; apenas termos TF-IDF e distribuição aproximada	Escolhido , sujeito a superar baseline e cumprir todos os RNF.

3.6 Ligação ao indicador de negócio

Se o MLP superar a baseline em Macro-F1 e encaminhar corretamente uma proporção útil dos acórdãos, a equipa espera reduzir o tempo de triagem inicial, sobretudo em lotes grandes. Contudo, o impacto real não pode ser deduzido apenas da Macro-F1. Deve ser medido num piloto: comparar o tempo e a taxa de correção manual de analistas com e sem o apoio do sistema, mantendo revisão humana. A magnitude da redução é uma **meta a validar com stakeholders**, não um resultado já demonstrado.

4. Data Preparation View — Fase 3

4.1 Fontes de dados e atributos

Fonte	Principais campos/atributos	Utilização
PDFs do CSM/portal de jurisprudência	nome do ficheiro, texto extraído, número de páginas, origem	Fonte bruta alternativa; passa pelo carregador PDF e parser posicional.
JSON bruto	string completa ou chave simples <code>texto</code>	Fonte bruta alternativa; é convertida em <code>DocumentoBruto</code> .
JSON estruturado irmão do PDF — source of truth	<code>descritores</code> , <code>sumario_texto</code> , <code>decisao</code> , <code>court_code</code> , <code>year</code> , <code>file_hash</code> , <code>extraction_success</code> e metadados	Rota principal para obter <code>Acordao</code> ; serve também como oráculo para validar o parser de PDF.
Manifesto e artefactos de execução	vocabulário, IDF, mapa de categorias, configuração do modelo, pesos e métricas	Garante que treino, avaliação e inferência usam o mesmo conjunto de artefactos.

4.2 Data Operations — transformações

Nº	Transformação	Fonte(s)
1	Descobrir ficheiros PDF/JSON e carregá-los incrementalmente com <code>yield</code> , registando falhas sem interromper o lote.	PDFs e JSONs brutos
2	Extrair o texto de todas as páginas do PDF ou ler o texto do JSON bruto, criando <code>DocumentoBruto</code> .	PDF/JSON bruto
3	Converter <code>DocumentoBruto</code> em <code>Acordao</code> por parsing posicional, ou converter diretamente JSON estruturado em <code>Acordao</code> .	<code>DocumentoBruto</code> /JSON estruturado
4	Separar <code>descritores</code> por <code>;</code> , normalizar espaços e tolerar campos ausentes/mojibake.	Campos estruturados
5	Construir o texto canónico de X exclusivamente com <code>descritores + sumario</code> .	<code>Acordao.texto_caracteristicas()</code>
6	Limpar ruído: marca TCPDF, emails, URLs, telefones, faxes, códigos postais, caracteres U+FFFD e espaços duplicados.	Texto canónico de X
7	Normalizar <code>decisao_bruta</code> para uma das cinco classes com regras ordenadas; descartar decisões vazias, ambíguas ou não reconhecidas.	<code>decisao_bruta</code>
8	Construir <code>RegistoClassificacao</code> com identificador, texto limpo, categoria e metadados de análise.	<code>Acordao</code> limpo
9	Dividir os registos de forma estratificada em treino e teste com semente 42.	<code>RegistoClassificacao</code>
10	Ajustar vocabulário e IDF apenas no treino ; transformar treino, teste e futura inferência com o mesmo vetorizador.	Textos de treino/teste

Nº	Transformação	Fonte(s)
11	Mapear as cinco categorias para IDs inteiros canônicos e criar matrizes NumPy.	Categorias normalizadas
12	Converter matrizes em tensores PyTorch, treinar configurações candidatas, avaliar, guardar pesos/configuração/métricas e manifesto.	Matrizes NumPy

4.3 Dataset final

Tabela conceptual antes da vetorização

Coluna	Descrição	Origem/transformação
id_documento	Identificador estável do registo	ECLI/hash/nome sanitizado; implementação de integração a consolidar
texto	Junção limpa de descritores e sumário	Acordao.texto_caracteristicas() + limpar_texto()
categoria_normalizada	Uma das cinco classes	normalizar_categoria(decisao_bruta)
tribunal	Metadado para análise de erros; nunca entra em X	court_code/ECLI
ano	Metadado para análise temporal; nunca entra em X	year/metadados

Tabela fictícia sanitizada

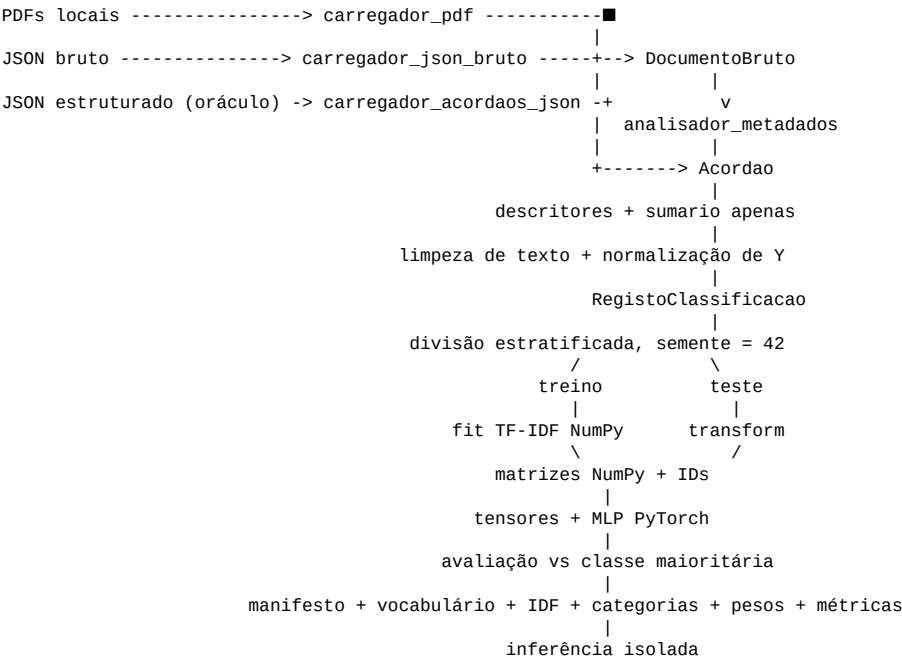
id_documento	texto (excerto sanitizado)	categoria_normalizada	tribunal	ano
DOC-001	recurso civil responsabilidade contratual tribunal aprecia nulidade...	ANULADA	TRL	2025
DOC-002	apelação contrato indemnização decisão recorrida fundamentos...	MANTIDA	TRP	2024
DOC-003	admissibilidade recurso prazo legal conhecimento do objeto...	NAO_CONHECIDA	STJ	2025

Os textos acima são fictícios e não reproduzem dados pessoais nem decisões reais.

Matrizes utilizadas pelo modelo

- `caracteristicas_treino`: matriz TF-IDF NumPy (`n_treino`, `n_tokens`).
- `caracteristicas_teste`: matriz TF-IDF NumPy (`n_teste`, `n_tokens`).
- `categorias_treino`: vetor NumPy de IDs inteiros.
- `categorias_teste`: vetor NumPy de IDs inteiros.

4.4 Diagrama do pipeline



4.5 Lacunas de integração identificadas

O desenho do pipeline está documentado e várias peças têm testes próprios, mas o repositório ainda apresenta sinais de integração incompleta: não existe um construtor explícito e versionado de `RegistoClassificacao`, o treino principal ainda usa dados simulados, e módulos esperados pelo motor de inferência (`manifesto.py` e uma implementação de classificador compatível) não estão presentes no estado inspecionado. Estas lacunas não invalidam o modelo GR4ML; tornam-se requisitos de integração e critérios prévios à demonstração ponta a ponta.

5. Frase de verificação final

Para ajudar o analista jurídico/investigador de jurisprudência a decidir como categorizar inicialmente cada acórdão e quando encaminhá-lo para revisão humana, respondendo à pergunta “qual das cinco classes melhor representa o sentido normativo deste documento, usando apenas descritores e sumário?”, vamos usar um MLP em PyTorch alimentado por TF-IDF implementado em NumPy, que satisfaz os softgoals de rigor em classes desequilibradas, ausência de fuga de informação, reprodutibilidade, privacidade, transparência e robustez, medidos por Macro-F1 face à baseline, auditoria dos campos de entrada, semente e manifesto, ausência de dados sensíveis externos, avisos de uso e continuidade após falhas, usando o dataset preparado de texto limpo de descritores + sumário e categoria normalizada, vindo de PDFs e JSONs de jurisprudência processados localmente. Isto deverá impactar o indicador tempo e consistência da triagem inicial, numa magnitude a medir e validar num piloto com utilizadores.

6. Dos modelos aos requisitos — Fase 4

6.1 Requisitos Funcionais

ID	Requisito Funcional	Origem GR4ML
RF-GR-01	O sistema deve carregar incrementalmente PDFs e JSONs locais, produzindo um contrato <code>DocumentoBruto</code> por ficheiro e continuando o lote quando um ficheiro falhar.	QG-04; pipeline Fase 3
RF-GR-02	O sistema deve converter cada fonte suportada num <code>Acordao</code> estruturado, extraindo ou adaptando descritores, sumário, decisão bruta e metadados disponíveis.	QG-01; entidades Fase 3
RF-GR-03	O sistema deve construir o texto de entrada exclusivamente a partir de <code>descritores</code> e <code>sumario</code> , aplicar a limpeza definida e manter a decisão verdadeira separada como rótulo.	QG-01; SG-02
RF-GR-04	O sistema deve normalizar decisões reconhecidas para as cinco classes canónicas e encaminhar decisões vazias, ambíguas ou não reconhecidas para descarte/revisão, sem adivinhação.	Decision Goal do analista; SG-05
RF-GR-05	O sistema deve criar registos de classificação, dividi-los de forma estratificada e ajustar o TF-IDF apenas no subconjunto de treino.	QG-03/QG-04; pipeline Fase 3
RF-GR-06	O sistema deve treinar e comparar pelo menos duas configurações do MLP PyTorch, conservando históricos e configurações para avaliação.	QG-05
RF-GR-07	O sistema deve avaliar o MLP no conjunto de teste e compará-lo com a baseline de classe maioritária, reportando pelo menos exatidão e Macro-F1.	QG-03; SG-01
RF-GR-08	O sistema deve guardar vocabulário, IDF, mapas de categorias, configuração, pesos, métricas e manifesto sob um único <code>id_execucao</code> .	QG-04; SG-03
RF-GR-09	O sistema deve receber um novo <code>Acordao</code> , usar a mesma composição e os mesmos artefactos do treino e devolver classe prevista e distribuição pelas cinco classes.	Decision Goal do analista; QG-01/QG-02
RF-GR-10	O sistema deve permitir exportar um resumo agregado das categorias previstas num lote, sem expor texto judicial ou dados pessoais no relatório público.	Goal do analista; SG-04

6.2 Requisitos Não-Funcionais

ID	Requisito Não-Funcional	Origem GR4ML
RNF-GR-01	O sistema deve garantir ausência de fuga de informação, medida por testes e auditoria, com zero utilização direta de <code>ecli</code> , <code>url</code> , <code>tribunal</code> , <code>texto_integral</code> ou <code>decisao_bruta</code> na matriz X.	SG-02

ID	Requisito Não-Funcional	Origem GR4ML
RNF-GR-02	O sistema deve avaliar dados desequilibrados com Macro-F1 e só considerar o MLP tecnicamente útil se superar a Macro-F1 da baseline no teste.	SG-01
RNF-GR-03	O sistema deve garantir reprodutibilidade básica com <code>semente = 42</code> , configuração, versões, plataforma e caminhos de artefactos registados por execução.	SG-03
RNF-GR-04	O sistema deve garantir integridade de artefactos: a inferência deve carregar todos os componentes pelo mesmo manifesto e nunca combinar ficheiros de execuções diferentes.	SG-03
RNF-GR-05	O sistema deve processar uma amostra de 10 documentos num computador modesto e continuar após uma falha isolada.	SG-06
RNF-GR-06	O sistema deve garantir privacidade: zero corpus real, texto integral, decisão bruta, dados pessoais, segredos, vocabulários ou artefactos identificáveis enviados para APIs externas ou incluídos no Git.	SG-04
RNF-GR-07	O sistema deve apresentar qualquer explicação como automática, aproximada e não jurídica; uma probabilidade softmax não pode ser chamada de confiança calibrada sem calibração própria.	SG-05
RNF-GR-08	O sistema deve manter código modular, funções públicas tipadas e testes <code>unittest</code> executáveis em Python 3.11.	Qualidade académica/técnica
RNF-GR-09	O parser deve tolerar campos jurídicos ausentes com <code>None</code> /lista vazia, sem transformar ausência de dados numa classificação inventada.	SG-05/SG-06
RNF-GR-10	A meta de latência da inferência em lote deve ser definida após medir o hardware-alvo; até essa validação, a execução deve registar duração e número de documentos.	SG-06; meta a validar

6.3 Requisitos de Dados

ID	Requisito de Dados	RF alimentado
RD-GR-01	O sistema deve ler PDFs locais e extrair texto e número de páginas incrementalmente, registando ficheiro, origem e falhas.	RF-GR-01/RF-GR-02
RD-GR-02	O sistema deve integrar JSONs estruturados com <code>extraction_success=true</code> , mapeando <code>descritores</code> , <code>sumario_texto</code> , <code>decisao</code> , <code>tribunal</code> , <code>ano</code> e proveniência para o contrato <code>Acordao</code> .	RF-GR-02
RD-GR-03	O sistema deve validar o parser de PDF contra o JSON irmão do mesmo documento, tratado como source of truth da extração.	RF-GR-02
RD-GR-04	O sistema deve dividir descritores por ponto e vírgula, remover itens vazios e normalizar espaços e caracteres corrompidos suportados.	RF-GR-03
RD-GR-05	O sistema deve remover do texto de X marcas TCPDF, emails, URLs, contactos, códigos postais e espaços redundantes antes da vetorização.	RF-GR-03/RF-GR-05
RD-GR-06	O sistema deve descartar do treino registos sem rótulo fiável e conservar os casos ambíguos para possível revisão, sem os forçar para <code>OUTRA</code> .	RF-GR-04/RF-GR-05
RD-GR-07	O sistema deve criar o vocabulário e os pesos IDF somente com os textos de treino e ignorar tokens desconhecidos em teste/inferência.	RF-GR-05/RF-GR-09
RD-GR-08	O sistema deve preservar <code>tribunal</code> e <code>ano</code> apenas como metadados para análise de erros, nunca como características de treino.	RF-GR-03/RF-GR-07
RD-GR-09	O sistema deve guardar os artefactos de dados e modelo com caminhos relativos num manifesto, associados a um único <code>id_execucao</code> .	RF-GR-08/RF-GR-09

6.4 Critérios de aceitação

Critério técnico mínimo da PoC

Este conjunto de requisitos será considerado tecnicamente bem-sucedido quando, numa execução reprodutível e auditável sobre dados separados, o MLP:

- superar a Macro-F1 da baseline de classe majoritária;
- usar apenas descritores e sumário como X;
- produzir todos os artefactos previstos num manifesto válido;
- realizar inferência isolada com a mesma transformação do treino;
- processar uma amostra de 10 documentos sem que uma falha individual interrompa o lote;
- manter os testes automatizados aprovados;
- não expor dados reais ou segredos.

Critério de valor para o utilizador — a validar

O benefício de negócio será considerado demonstrado se, num piloto controlado, analistas jurídicos reduzirem o tempo mediano de triagem inicial **sem aumentar a taxa de categorização incorreta após revisão humana**. O valor atual, a meta e o período de observação devem ser acordados com os utilizadores antes de uma alegação de impacto.

7. Reflexão crítica

O ML não é estritamente necessário para todas as partes deste problema. A extração de metadados, a limpeza, a normalização de algumas decisões e a criação de relatórios podem ser resolvidas por regras determinísticas. Para um corpus pequeno, a classificação manual ou uma pesquisa por palavras-chave também poderia ser suficiente e mais fácil de explicar. O ML torna-se justificável quando o volume cresce, a linguagem varia e o objetivo é priorizar ou organizar milhares de documentos de forma consistente — desde que exista revisão humana e que o modelo seja realmente melhor do que uma regra simples ou a classe majoritária.

A formalização dos softgoals revelou vários pontos que uma conversa inicial centrada apenas em “classificar acórdãos” poderia esconder. Primeiro, a exatidão não é uma medida suficiente porque **MANTIDA** domina o corpus; a Macro-F1 e a baseline são requisitos de negócio técnico, não detalhes opcionais. Segundo, uma boa métrica seria inválida se a decisão, o ECLI ou outro atalho entrasse nas características. Terceiro, “explicabilidade” precisava de ser qualificada: mostrar termos TF-IDF e probabilidades ajuda a inspecionar a entrada e a saída, mas não constitui fundamentação jurídica causal nem confiança calibrada. Quarto, privacidade e proveniência dos artefactos fazem parte do produto de ML; um vocabulário TF-IDF também pode revelar informação.

A tradução para requisitos tornou ainda mais clara a lacuna entre módulos testados isoladamente e um sistema ponta a ponta. O repositório documenta o fluxo pretendido, mas a integração precisa de materializar o construtor de registos, substituir dados simulados por matrizes reais, produzir o manifesto completo e alinhar as interfaces esperadas pela inferência. Portanto, o principal valor do GR4ML neste projeto foi impedir que “treinar uma rede” fosse confundido com concluir o sistema. O sucesso depende igualmente de dados, contratos, avaliação, segurança, operação, supervisão humana e evidência de impacto.

Rastreabilidade principal para o repositório

- Objetivo e características autorizadas: [README.md:3-12](#).
- PoC, viés, transparência e RGPD: [docs/visao.md:3-12](#) e [docs/etica.md:3-21](#).
- RF/RNF existentes: [docs/requisitos.md:5-30](#).
- Pipeline e artefactos: [docs/arquitetura.md:5-50](#).
- Contratos e fonte canónica de X: [src/dados/esquemas.py:20-101](#).
- Source of truth, campos e distribuição: [docs/esquema_json_corpus.md:3-123](#).
- Decisões arquiteturais: [docs/decisooes.md:5-32](#).
- Ingestão incremental e tolerante a falhas: [src/dados/carregador_pdf.py:23-84](#) e [src/dados/carregador_json_bruto.py:22-86](#).
- Parsing posicional: [src/pre_processamento/analizador_metadados.py:159-232](#).
- Limpeza e normalização de Y: [src/pre_processamento/limpeza_texto.py:22-152](#).
- Divisão e TF-IDF NumPy: [src/caracteristicas/vetorizador_tfidf.py:33-395](#).
- MLP e treino: [src/modelos/rede_neuronal.py:10-41](#) e [src/treino/treinar_modelo.py:22-246](#).
- Baseline e Macro-F1: [src/avaliacao/metricas.py:19-116](#).
- Inferência e avisos: [src/inferencia/motor_inferencia.py:24-201](#) e [src/inferencia/formatador_saida.py:17-204](#).